UNIVERSITY OF ENGINEERING AND TECHNOLOGY, PESHAWAR

Department of Computer Science

DATA STRUCTURE LAB

LAB TASK 6

Singly Linked List: Insertion at Start, End, and Before a Target Value

Submitted By:	Anas Khan
Registration No.:	25PWCS1691
Section:	B
Semester:	2nd
Department:	Computer Science
Submitted To:	Sir Mudasir
Date:	21 October 2026

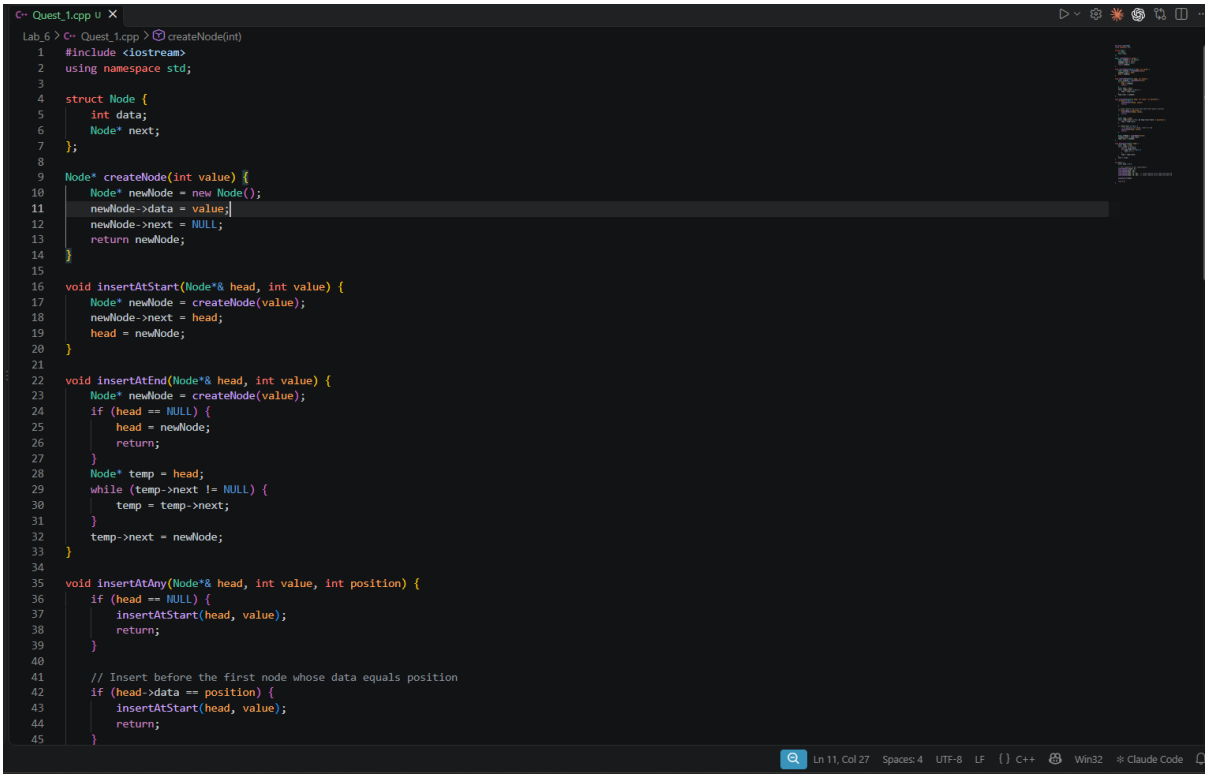
1. Task Statement

Write one C++ program for a singly linked list that supports three insertion operations:

- insertAtStart(head, value) inserts a new node at the beginning of the list.
- insertAtEnd(head, value) inserts a new node at the end of the list.
- insertAtAny(head, value, position) inserts a new node before the first node whose data equals position.

The program also includes displayList(head), which prints all node values from head to NULL. It must run the required test sequence and produce the final list 5 10 15 20 30.

2. Program



```cpp
    // Insert before the first node whose data equals position
    if (head->data == position) {
        insertAtStart(head, value);
        return;
    }

    Node* temp = head;
    while (temp->next != NULL && temp->next->data != position) {
        temp = temp->next;
    }

    if (temp->next == NULL) {
        // If position not found, insert at end
        insertAtEnd(head, value);
        return;
    }

    Node* newNode = createNode(value);
    newNode->next = temp->next;
    temp->next = newNode;
}

void displayList(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data;
        if (temp->next != NULL) {
            cout << " ";
        }
        temp = temp->next;
    }
    cout << endl;
}

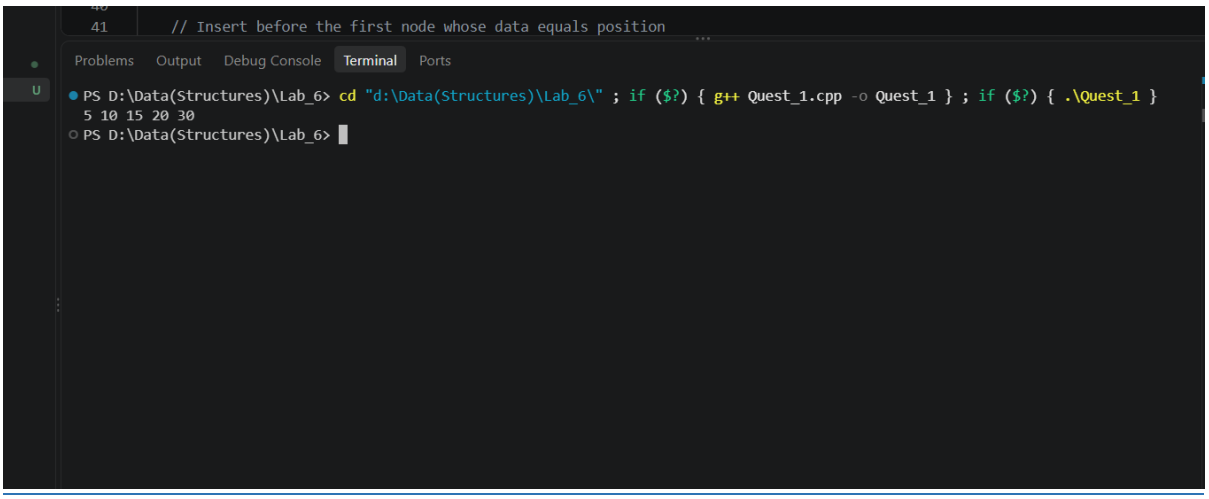
int main() {
    Node* head = NULL;

    // Test sequence as per requirements
    insertAtStart(head, 5);
    insertAtEnd(head, 10);
    insertAtEnd(head, 30);
    insertAtAny(head, 15, 30);  // insert before first node with data 30
    insertAtAny(head, 20, 30);  // insert before first node with data 30

    displayList(head);

    return 0;
}
```


Screenshot



4. Trace

Test Sequence

Step	Operation	List after step
1	`insertAtEnd(head, 10)`	10
2	`insertAtEnd(head, 20)`	10 20
3	`insertAtStart(head, 5)`	5 10 20
4	`insertAtAny(head, 15, 20)`	5 10 15 20
5	`insertAtEnd(head, 30)`	5 10 15 20 30
6	`displayList(head)`	5 10 15 20 30

After Step 3: Where does head point?

head points to the node containing 5. The new node was linked to the old head (10), and then head was moved to the new node.

```
head -> [5|*] -> [10|*] -> [20|NULL]
```

During Step 4: insertAtAny(head, 15, 20)

- New node = 15
- Target node = 20
- current stops at node 10, because current->next->data == 20

Before:

```
head -> [5|*] -> [10|*] -> [20|*] -> NULL
                    ^           ^
                 current     target (20)
```

Step	Action	Result
1	`head->data (5) != 20, so current = head`	current = 5
2	current->next->data is 10, not 20, move forward	current = 10
3	current->next->data is 20, match found, stop	current = 10
4	newNode->next = current->next	15 points to 20
5	current->next = newNode	10 points to 15

After:

```
head -> [5|*] -> [10|*] -> [15|*] -> [20|*] -> NULL
                             ^
                          new node
```

During Step 5: insertAtEnd(head, 30)

List before: 5 -> 10 -> 15 -> 20 -> NULL

Move	current	current->next != nullptr ?

Start	5	Yes, move on
1	10	Yes, move on
2	15	Yes, move on
3	20	`No (20->next is NULL), stop`

Then current->next = newNode links 20 to 30.

```
head -> [5] -> [10] -> [15] -> [20] -> [30] -> NULL
```